Week 4 · Functions you can trust · Midterm

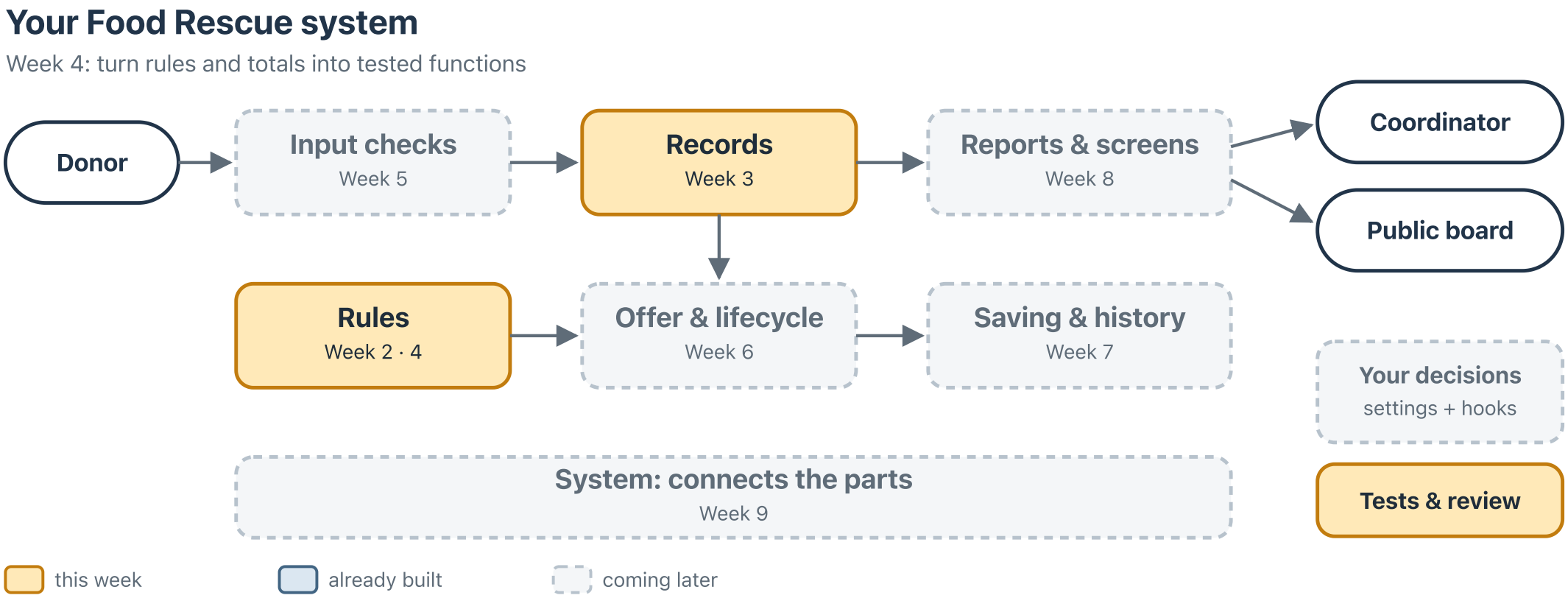
Programming for Information Systems

A clear promise, one returned answer, and tests that prove it

This week

Question	How can you show that a function keeps its promise?
Python	`def`, parameters, `return` vs `print`, `None`, docstrings, `assert`, test functions
Principle 4	One function, one job: return values, test normal, boundary and odd cases
You build	`open_portions` and your own tests
Session 3	the individual midterm

Your system, by Week 10



The idea: modules and contracts

A phone charger promises: wall power in, safe power out, **never** damages the phone.

A function's **contract** says:

- **Inputs:** what you must give it.
- **Output:** what it returns (and when it returns `None`).
- **Never:** what it must not do, e.g. change the list you give it, or print.

> A **module** is a file of functions with contracts. Parts that keep their contracts can be swapped safely. A **test** shows the contract is kept.

Python foundations

Functions you can trust

Parameters and arguments

```
def price_after_discount(price_riel, discount_percent):
    return price_riel - price_riel * discount_percent // 100

print(price_after_discount(10000, 10))     # 9000
print(price_after_discount(discount_percent=25, price_riel=4000))
print(price_after_discount(10, 10000))     # 🤔 ?
```

- **Parameters**: the names in the `def` line. **Arguments**: the values in the call.
- Matched by **position**, unless you give them by name.

`return` or `print`?

```
def total_printed(prices_riel):
    total = 0
    for price_riel in prices_riel:
        total += price_riel
    print(total)

printed_result = total_printed([4000, 1500])   # shows 5500
print(printed_result)                          # None
```

- `print` **shows** a value to a person, then it is gone.
- `return` **gives** the value back, so code can add, compare and test it.

`None` means "no answer"

```
prices_by_item = {"notebook": 4000, "pen": 1500}

def find_price(item_name):
    return prices_by_item.get(item_name)

price = find_price("ruler")
if price is None:
    print("Not sold here")
```

- `None` **on purpose** is part of the contract: write it down.
- `None` **by accident** (a forgotten `return`) is a bug.

Docstrings: write the promise down

```
def average_mark(marks):
    """Return the average of a list of marks.

    Returns 0 for an empty list. Does not change the list.
    """
    if len(marks) == 0:
        return 0
    ...

help(average_mark)
```

- The first text inside a function, in triple quotes.
- Say the **output**, the **odd cases**, and what it **never** does.

Functions that call functions

```
URGENT_HOURS = 3

def is_urgent(hours_left):
    return hours_left <= URGENT_HOURS

def urgent_portions(records):
    total = 0
    for record in records:
        if is_urgent(record["hours_left"]):
            total += record["portions"]
    return total
```

- Each function does **one job** and can be tested alone.

`assert`: a check that stops loudly

```python
assert price_after_discount(10000, 10) == 9000
assert average_mark([]) == 0
print("Both asserts passed")
# assert average_mark([50, 70]) == 50   # AssertionError
```

- `True` → nothing happens. `False` → Python stops with `AssertionError`.
- A broken promise should never pass quietly.

Test functions: normal, boundary, empty

```
def test_average_normal():
    assert average_mark([60, 80]) == 70

def test_average_boundary():
    assert average_mark([0, 100]) == 50

def test_average_empty():
    assert average_mark([]) == 0

run_tests([test_average_normal, test_average_boundary, test_average_empty])
```

- Name starts with `test_`; one case per test; uses `assert`.
- `run_tests` (supplied) prints ✅ or ❌ for each test. Give it the functions **without** `( )`.

Round up, and test with the constant

```
SEATS_PER_MINIBUS = 12

def minibuses_needed(students):
    return students // SEATS_PER_MINIBUS + 1   # a tempting shortcut

print(minibuses_needed(25))   # 3  ✓
print(minibuses_needed(24))   # 3  ✗ exactly two full buses
print(minibuses_needed(0))    # 1  ✗
```

- Round up with `//` (full buses) **and** `%` (anyone left over?), from Week 1.
- Test with the constant: `minibuses_needed(SEATS_PER_MINIBUS + 1) == 2`, not `(13) == 2`. Change it to 15: a raw `13` breaks. It was a **magic number**.

⚖️ Good or risky?

Version A (from an AI assistant)

```
def delivered_portions_a(records):
    total = 0
    for record in records:
        if record["status"] == "delivered":
            total += record["portions"]
            records.remove(record)
    print("Delivered:", total, "portions")

def test_delivered_a():
    delivered_portions_a(fr.sample_records())
```

Version B

```
def delivered_portions_b(records):
    """Return delivered portions.
    Does not change records."""
    total = 0
    for record in records:
        if record["status"] == "delivered":
            total += record["portions"]
    return total

# 4 tests: normal, two in a row,
# empty, keeps the list
```

Principle 4 · One function, one job

- **One job:** calculate, *or* show, *or* change. Not all three.
- **Return** the value; let the caller decide whether to print it.
- **Never change the input** unless that is the function's one job.
- **Test** normal, boundary and odd (empty) cases.

> When you review AI code, ask: *what does this test actually check?*

You build: `open_portions` and four tests

```
OPEN_STATUSES = ["new", "approved", "picked_up"]

def open_portions(records):
    """Return the total portions of records whose status is in OPEN_STATUSES.

    Returns 0 for an empty list. Does not change records and does not print.
    """
    return None   # TODO

fr.check("open_portions", open_portions)     # sample → 42, [] → 0
```

- Your tests: **normal** (42), **boundary** (picked up counts, delivered does not), **empty** (0), **keeps the list**.

Plug in: does it agree all day?

```
compare("End of the sample day")      # 42
my_system.add_offer({... "portions": "16" ...})
compare("After a new offer arrives")  # 58
my_system.move("F001", "delivered")
compare("After F001 is delivered")    # 28
```

- `my_system.numbers()["portions_open"]` is the system's own number.
- A function you can trust agrees **after every change**, not only on the sample.

🧭 Your decision · which statuses count as open?

No new setting: the decision lives in `OPEN_STATUSES`.

1. Should **picked up** count? The food is in a basket, not yet with the partner.
2. Should **new** count? The coordinator may still reject it.
3. Who reads this number, and what do they do with it?

> Keep the system's definition for the check. Write your reasoning in the decision log.

Session 3 · Midterm · Plan the day

Part	What you do	Points
A · Read code	explain what five snippets print, and **why**	20
B · Write a rule	a small function with a named constant	20
C · Work with records	summarise a list of offers without changing it	25
D · Tests that catch bugs	four tests, run against planted bugs	15
E · Judge AI code	find problems in code that runs	10
F · Your decision	choose a constant, restart and run all, decision log	10

- **Individual.** Allowed: your notes, the weekly notebooks, Python Basics. Not allowed: AI tools, messaging.
- Then a **3-minute oral check**: explain one line of your code and one test.

This week's tasks

- ⭐ **Core:** P1–P4, three problems in Version A, `open_portions` with ✅, four passing tests, agreeing with the system, decision log.
- ⭐⭐ **Stretch:** S1 (a test that catches a bug); `portions_with_status` with your tests still passing.
- 📝 **Session 3:** the midterm in `Midterm.ipynb`.
- 🔎 **Research 3 (5%), set after the midterm:** checking input on a real form. Share it in the Week 5 warm-up.

Next week · Checking input

Real input is messy: extra spaces, words instead of numbers, missing fields. You will check **every offer at the door**, and change nothing when input is bad.